

Compiler Construction Tools

CSE 4802 Compiler Lab

Flex/Yacc installation

Windows:

Install *Flex Windows [Lex and Yacc].exe* with administrative privilege
Run the program with administrative privilege

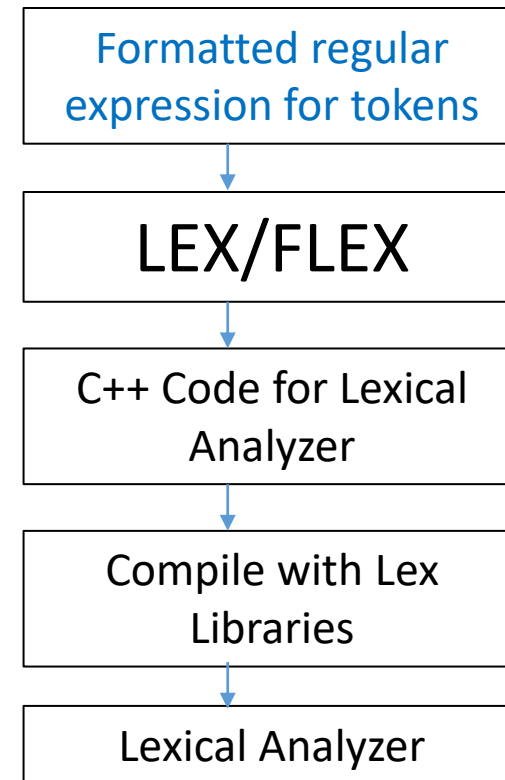
Linux:

Install *lex*, *bison* and *gcc* packages.

Lex

Lex is a compiler construction tool. It helps to generate Lexical Analyzer.

Flex: Fast Lex



Lex Program Structure

A lex program should have three sections separated by two set of “%%” strings.

Declarations

%%

Rules (regular expressions and associated actions)

%%

Subroutines

Build Lex Program in Linux

Let *count.l* is a lex program.

```
lex count.l
```

Above command compile the lex program and on success it creates a C equivalent file named *lex.yy.c*

```
gcc -lfl lex.yy.c
```

Upon success above command creates an executable file named *o.out*

```
./o.out
```

```
gcc -lfl lex.yy.c -o count
```

Regular Expression

<i>Metacharacter</i>	<i>Matches</i>
.	any character except newline
\n	newline
*	zero or more copies of preceding expression
+	one or more copies of preceding expression
?	zero or one copy of preceding expression
^	beginning of line
\$	end of line
a b	a or b
(ab) +	one or more copies of ab (grouping)
"a+b"	literal "a+b" (C escapes still work)
[]	character class

Regular Expression

<i>Expression</i>	<i>Matches</i>
<code>abc</code>	<code>abc</code>
<code>abc*</code>	<code>ab, abc, abcc, abccc, ...</code>
<code>abc+</code>	<code>abc, abcc, abccc, ...</code>
<code>a(bc) +</code>	<code>abc, abcbc, abcbcbc, ...</code>
<code>a(bc) ?</code>	<code>a, abc</code>
<code>[abc]</code>	<code>a, b, c</code>
<code>[a-z]</code>	any letter, a through z
<code>[a\ -z]</code>	<code>a, -, z</code>
<code>[-az]</code>	<code>-, a, z</code>
<code>[A-Za-z0-9] +</code>	one or more alphanumeric characters
<code>[\t\n] +</code>	whitespace
<code>[^ab]</code>	anything except: <code>a, b</code>
<code>[a^b]</code>	<code>a, ^, b</code>
<code>[a b]</code>	<code>a, , b</code>
<code>a b</code>	<code>a or b</code>

Some Lex variable and functions

<i>name</i>	<i>function</i>
<code>int yylex(void)</code>	call to invoke lexer, returns token
<code>char *yytext</code>	pointer to matched string
<code>yylen</code>	length of matched string
<code>yylval</code>	value associated with token
<code>int yywrap(void)</code>	wrapup, return 1 if done, 0 if not done
<code>FILE *yyout</code>	output file
<code>FILE *yyin</code>	input file
<code>INITIAL</code>	initial start condition
<code>BEGIN condition</code>	switch start condition
<code>ECHO</code>	write matched string